

CC1002 Introducción a la Programación - Sección 1

Tarea 2

Prof. Benjamin Bustos

Fecha de entrega: viernes 9 de octubre de 2026 a las 23:59

Es obligatorio el uso de la receta de diseño visto en cátedra para implementar todas las funciones.

La tarea debe realizarse en forma individual. No olvide leer el documento de Política de Colaboración publicado en U-Cursos.

Simulando una cola de espera

Un uso importante de la computación es para simular diferentes aspectos de la vida real, de forma de poder estudiarlos. Por ejemplo, los científicos simulan en el computador cómo interactúan distintos tipos de proteínas, cómo el tráfico fluye dentro de una ciudad, cómo se comporta el caudal de un río, etc. En particular, la *teoría de colas* corresponde al estudio de las líneas de espera de un sistema, por ejemplo, clientes que ingresan a una cola y esperan a ser atendidos por una caja. Esto sirve, por ejemplo, para estudiar cuánto tiempo debe esperar una persona desde que ingresa a la cola hasta ser atendido por una caja, cómo se modifica dicho tiempo si se agregan o quitan cajas, o qué pasa si en vez de una cola de espera por caja se utiliza una única cola de espera. También se pueden simular sistemas de comunicaciones, en donde los elementos que entran a las colas de espera son paquetes de datos y las “cajas” corresponden a máquinas que deben procesar o transmitir estos datos.

Un aspecto importante para el estudio de las líneas de espera usando el computador es simular la cola, la cuál puede tener distintas características o distintas funcionalidades. En una versión simple, la cola permite que una persona ingrese a ella, o que salga de la cola la siguiente persona a ser atendida. Esto puede sufrir variaciones dependiendo de cuántas colas de espera

tiene el sistema, el largo de las colas, cuánto tiempo se demora cada persona en ser atendida, si hay personas que tienen prioridad en ser atendidas, etc.

En esta tarea supondremos un escenario de simulación ficticio, con una única cola de espera que cumpla con las siguientes características:

- La cola guarda datos de tipo String.
- La cola mantiene la cantidad de datos que están almacenados en ella.
- La cola permite que un dato ingrese a la cola, ubicándose en la última posición.
- La cola permite que el dato que está en la primera posición de la cola (el primero que entró) salga de la cola.
- La cola permite que el dato que está en la última posición de la cola (el último que entró) salga de la cola.

Representación de la cola de espera

Para representar la cola de espera, utilizaremos el siguiente dato compuesto (**ESTA ESTRUCTURA NO PUEDE MODIFICARSE**):

```
# colaEspera: lista(str) lista(str) lista(str) int int str
estructura.crear('colaEspera', 'ultimos auxiliar primeros cantidadUltimos cantidadPrimeros valor')
```

Este dato compuesto permite guardar la información de la cola de espera usando tres listas recursivas no-mutables (utilice el módulo `lista.py` publicado en el Material Docente) y algunos atributos adicionales:

- La lista `ultimos` guarda los últimos datos ingresados (los que están al final de la cola) en orden *inverso* de ingreso (la cabeza de esta lista corresponde al último dato ingresado).
- La lista `primeros` guarda los primeros datos ingresados (los que están al principio de la cola) en orden de ingreso (la cabeza de esta lista corresponde al primer dato ingresado).
- La lista `auxiliar` en general está vacía, sólo se ocupa si es necesario *balancear* la cola de espera (ver explicación más adelante).
- El entero `cantidadUltimos` contiene la cantidad de datos almacenados en la lista `ultimos`.

- El entero `cantidadPrimeros` contiene la cantidad de datos almacenados en la lista `primeros`.
- El String `valor` corresponde al último dato que se ingresó o sacó de la cola de espera.

Al crear una cola de espera, las tres listas están vacías, la cantidad de elementos por lista es cero, y el atributo `valor` se inicializa con el String vacío (`""`).

Para implementar las operaciones de la cola de espera, considere lo siguiente (**NO SE PUEDEN MODIFICAR ESTOS ALGORITMOS**):

- Ingresar un dato en la cola de espera: se añade a la cabeza de la lista `ultimos`, se actualiza la cantidad de elementos en esta lista y se actualiza el atributo `valor`.
- Sacar el primer dato de la cola de espera: se extrae la cabeza de la lista `primeros`, se actualiza la cantidad de elementos en esta lista y se actualiza el atributo `valor`.
- Sacar el último dato de la cola de espera: se extrae la cabeza de la lista `ultimos`, se actualiza la cantidad de elementos en esta lista y se actualiza el atributo `valor`.
- Para estas últimas dos operaciones, en caso que al intentar sacar un dato la lista correspondiente está vacía, primero se debe *balancear* la cola de espera con la siguiente estrategia:
 - Sea n la cantidad total de datos en la cola de espera (note que están todos en una única lista, `primeros` o `ultimos`, que denominaremos *original*).
 - Se sacan (uno a uno) $\lfloor \frac{n}{2} \rfloor$ datos de la lista original y se van insertando en la lista `auxiliar`.
 - Se sacan (uno a uno) el resto de los datos de la lista original y se van insertando en la lista que estaba vacía (note que de esta forma quedan en el orden correcto).
 - Se sacan (uno a uno) todos los datos de la lista `auxiliar` y se van insertando de vuelta en la lista original.
 - Se actualiza la cantidad de elementos en las listas `primeros` y `ultimos`.

Notar que no se puede sacar un dato si la cola de espera está vacía, esto puede provocar que el programa se caiga.

Tarea

Implemente un módulo de Python que provea las siguientes funciones:

1. `ingresar(dato, unaCola)`: Función que recibe un dato y una cola de espera, y devuelve una cola de espera donde se ingresó el dato a la cola, siguiendo el proceso explicado previamente.
2. `primero(unaCola)`: Función que recibe una cola de espera (debe tener al menos un dato), y devuelve una cola de espera donde se sacó el primer dato de la cola de espera, siguiendo el proceso explicado previamente.
3. `ultimo(unaCola)`: Función que recibe una cola de espera (debe tener al menos un dato), y devuelve una cola de espera donde se sacó el último dato de la cola de espera, siguiendo el proceso explicado previamente.
4. `balance(unaCola)`: Función que recibe una cola de espera (debe tener al menos un dato), y devuelve una cola de espera después de realizar la operación de balance, siguiendo el proceso explicado previamente.
5. `colaEspera2texto(unaCola)`: Función que recibe una cola de espera, y devuelve un String con los valores de las listas `ultimos` y `primeros` separados por `->`, además de mostrar el String del atributo `valor`. Por ejemplo, si a partir de una cola de espera vacía se insertaron los datos 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', y luego se saca el primero, la función `colaEspera2texto` invocada con dicha cola de espera devuelve el siguiente String:

```
Ultimos: h -> g -> f -> e
Primeros: b -> c -> d
Valor: a
```

Ud. debe implementar en forma obligatoria en su módulo todas estas funciones solicitadas. Además, Ud. debe implementar todas las funciones auxiliares que requiera, en caso por ejemplo que para implementar algunas de las funciones solicitadas necesite descomponer el problema en subproblemas más pequeños. Recuerde que cada función implementada debe resolver un único problema o realizar una única tarea, si está intentando resolver más de un problema a la vez piense en cómo se podría descomponer en subproblemas (esto será evaluado).

Finalmente, implemente en otro módulo un programa interactivo que cree una cola de espera vacía, y que luego pregunte cuál operación se desea realizar. Si se desea ingresar un dato a la cola de espera, se debe preguntar por este dato. Cada vez que se ingresa o saca un dato (ya sea el primero o el último), se debe mostrar en pantalla el estado de la cola luego de la operación, y luego volver a preguntar por una operación. En el módulo del programa interactivo importe el módulo con las funciones programadas y utilícelo. Además, en este módulo puede agregar funciones o procedimientos auxiliares que pudiese requerir.

Ejemplo de lo que muestra el programa interactivo:

```
Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 1
Valor? A
Ultimos: A
Primeros:
Valor: A
```

```
Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 1
Valor? B
Ultimos: B -> A
Primeros:
Valor: B
```

```
Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 1
Valor? C
Ultimos: C -> B -> A
Primeros:
Valor: C
```

```
Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 1
Valor? D
Ultimos: D -> C -> B -> A
Primeros:
Valor: D
```

```
Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 1
Valor? E
Ultimos: E -> D -> C -> B -> A
Primeros:
Valor: E
```

Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 2
Ultimos: E -> D
Primeros: B -> C
Valor: A

Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 2
Ultimos: E -> D
Primeros: C
Valor: B

Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 1
Valor? F
Ultimos: F -> E -> D
Primeros: C
Valor: F

Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 1
Valor? G
Ultimos: G -> F -> E -> D
Primeros: C
Valor: G

Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 2
Ultimos: G -> F -> E -> D
Primeros:
Valor: C

Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 2
Ultimos: G -> F
Primeros: E
Valor: D

Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 3
Ultimos: F
Primeros: E
Valor: G

Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 3
Ultimos:

Primeros: E

Valor: F

Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)? 3

Ultimos:

Primeros:

Valor: E

Op (1:ingresar, 2:sacar primero, 3:sacar ultimo)?

Para el programa interactivo, ignore cualquier operación ingresada que no sea un valor 1, 2 o 3.

Testing

Debe implementar al menos cuatro tests para toda función programada en el módulo solicitado, incluyendo cualquier función auxiliar que haya implementado. No es necesario incluir testing para el programa interactivo ni para los procedimientos que requiera implementar.

Asignación de puntajes

- Receta de diseño: 1.5 puntos (repartido entre todas las funciones solicitadas).
 - Contrato: 0.4 ptos. en total
 - Descripción: 0.15 ptos. en total
 - Ejemplo: 0.15 ptos. en total
 - Testing: 0.8 ptos. (cuatro tests en cada una de las funciones programadas)
- Funciones solicitadas para módulo: 4.0 ptos en total.
 - Función `ingresar`: 0.4 ptos.
 - Función `primero`: 0.4 ptos.
 - Función `ultimo`: 0.4 ptos.
 - Función `balance`: 2.0 ptos.
 - Función `colaEspera2texto`: 0.8 ptos.

- Módulo programa interactivo: 0.5 pts.
- Descuentos globales:
 - Uso de buenas metodologías de programación: hasta -0.5 pts.
 - Descomposición funcional: hasta -0.5 pts.

Planificación para realizar la tarea

Hay cuatro semanas de plazo para implementar todo lo solicitado en esta tarea. Se le recomienda la siguiente planificación para ir avanzando parcialmente en la tarea cada semana:

- Leer y entender bien el enunciado: Semana 1.
- Diseñar una descomposición funcional adecuada para todas las funciones solicitadas: Semana 2.
- Módulo con funciones: Semanas 2 a 4.
- Programa interactivo: Semanas 3 y 4.

Notas importantes

- Para programar su tarea, sólo puede utilizar las herramientas de Python que hayamos visto en el curso. En particular, no se pueden usar instrucciones de Python que no se hayan visto en las clases de cátedra correspondientes las Unidades I y II del curso, no estén en el apunte entre los Capítulos 1 a 10, o no hayan sido especificadas en este enunciado. Soluciones que no cumplan con este punto, serán evaluadas con la nota mínima.
- Se evaluará el uso de buenas metodologías de programación (receta de diseño, nombres de funciones y variables representativos, evitar duplicar código, etc.).
- Antes de ponerse a programar, piense en cuál sería una descomposición funcional adecuada para cada función solicitada.
- Esta prohibido utilizar ambientes de desarrollo con IA integrada, por ejemplo Colab. Utilice IDLE que se instala junto con Python para programar su tarea.

- Implemente el módulo solicitado en el archivo `moduloTarea2.py`, y el programa interactivo en el archivo `programaTarea2.py`. No es necesario incluir los módulos `estructura.py` y `lista.py` del curso en su entrega.
- Su tarea debe respetar *estrictamente* los nombres de funciones, parámetros, tipos de datos y reglas descritos en el enunciado.
- No olvide leer y seguir las reglas descritas en el documento de Política de Colaboración publicado en U-Cursos.
- Dudas sobre el enunciado pueden postearlas en el foro del curso. *ESTÁ PROHIBIDO COPIAR CÓDIGO FUENTE DE SU TAREA EN EL FORO*. Si Ud. copia código de su tarea en el foro, aunque sea para preguntar por un error, tendrá nota 1.0.
- La tarea debe entregarse a través de la página de U-Cursos, sección Tareas. Entregas durante el período de entrega con atraso no tienen descuento. *NO SE ACEPTARÁN TAREAS FUERA DE PLAZO NI ENTREGADAS POR OTROS MEDIOS*.